

Mizan.ai

VAT Compliance Center — Build & Findings

What it offers, how invoice compliance checking and period VAT return preparation work end to end, the real issues found through live testing, and how each was fixed.

1. What this replaces, and why

Calculator was originally a single-invoice wizard: upload one filing's documents, extract, correct, enter line items, confirm, get a report. Working through real use cases surfaced that this doesn't match either job a VAT compliance tool actually needs to do:

- **Checking invoice compliance** is a batch job in practice — a pile of vendor invoices at month-end, not one invoice worked through interactively.
- **Preparing a period VAT return** is a fundamentally different shape of input entirely — a whole period's sales and purchases (many transactions, many counterparties), not a single seller/buyer/date.

The rebuild replaces the wizard with **VAT Compliance Center**: one home-page card, two connected, batch-oriented flows — referred to below as Job 1 and Job 2. Job 2 leans on Job 1's machinery wherever it can, rather than duplicating it.

2. Job 1 — Check invoices for compliance

Real-world purpose: mainly for invoices *received* from vendors — before trusting them and reclaiming VAT on them, not re-checking invoices your own system already generated correctly. Verifying structural validity and correct calculation, at batch scale.

Why this is genuinely useful — and where it isn't

This was deliberately not assumed — the real question asked and answered before building it: do people actually need this, or does ZATCA's e-invoicing mandate already make it redundant?

- **The value is concentrated on the receiving side.** Saudi Arabia's Phase 2 e-invoicing integration means ZATCA validates an invoice's structure and required fields at the point the ISSUER submits it for clearance — that protects the issuer's own outgoing invoices, generated by their own compliant system. It says nothing about invoices a business RECEIVES from someone else, where there is no such guarantee.
- **The real stakes:** claiming input VAT on a vendor invoice that turns out not to be compliant can have that claim disallowed in a future ZATCA audit — real money, not a formatting nitpick. That risk is exactly what checking incoming invoices before trusting them protects against.
- **Where it's largely redundant:** re-checking a business's own freshly-generated outgoing invoices, already produced by a compliant system and already going through ZATCA clearance, is checking work that's already been checked. That case was deliberately not the framing this feature was built around.
- **Matches the real usage pattern, not an idealized one:** checking happens in batches — at month-end before filing, or when a new or lower-trust vendor's invoices need establishing a pattern — not invoice-by-invoice as each one arrives. That's the direct reason Job 1 is a batch tool rather than a single-invoice checker (see §4's "batch-first" decision).

Aspect	Detail
Input	One or more invoice files — ZATCA e-invoice XML, or a PDF with an embedded XML attachment. Multi-file select, no directory picker.
Processing	Fully automatic — no human input mid-batch. Each invoice: extract → structural checks → independent VAT recompute vs. printed totals → clean/flagged decision.
Narration	Generated only for flagged invoices — a clean invoice never triggers a model call, since nothing would be narrated.
Output	One ZIP: <code>summary.<format></code> (every invoice + status), plus one <code><file>_issues.pdf</code> per FLAGGED invoice only — clean invoices produce no file.
Formats	xlsx / pdf / docx for the summary; detail files are always PDF (a narrative, not a data table).

Algorithm

- Extract via the doc-extraction service (not in-process — see §5).
- If no line items came back (not a real invoice document), flag as unreadable, skip further checks.
- Structural checks: seller VAT number format (15 digits, starts/ends '3'), issue date sanity (not in the future), seller/buyer VAT numbers not identical — reused from the constituent pieces of `structural_rules.py`, not its form-oriented orchestrator (see §5).
- VAT recompute: `tools.validate_zatca_form()` called directly, in-process, no model or subprocess — cheap enough to run on every invoice in the batch.
- Status = flagged if any structural issue is an error, or the recompute doesn't match what's printed.
- Narration (only if flagged): one direct model call given every finding together (see §5's narration fix).

3. Job 2 — Prepare a period VAT return

Real-world purpose: the actual month/quarter-end task — turning a period's sales and purchase registers into the Output VAT / Reclaimable Input VAT / Net position figures a real ZATCA return needs.

Aspect	Detail
Input	One multi-sheet workbook covering sales + purchases, or separate single-sheet files — treated identically.
Role mapping	Every sheet/file is extracted and role-guessed (sales / purchases / unknown) from its reclaimable-column presence, filename, and title text — but NEVER trusted silently. Always shown for the user to confirm or correct.
Per-row checks	Recompute-and-compare against the register's own declared total row(s), where a tax-category column lets a rate be inferred.
Reclaimable VAT	Priority order: (1) trust the register's own stated reclaimability if present, (2) tier-1 rule-based auto-exclude, (3) tier-2 — genuinely ambiguous, collected and shown together for one human decision.
Output	One ZIP: <code>vat_return_filing_<period></code> (headline figures only) + <code>vat_return_workpapers_<period></code> (headline + every exclusion with its reason + consistency findings).
Formats	xlsx / pdf / docx, same set as Job 1.

Algorithm

- Extract every uploaded file/sheet via the same doc-extraction service call; each sheet becomes its own "source", carrying its own column names and the title/metadata text sitting above its real header row.
- Role guess signals, in priority order: a reclaimable-type column present (strong purchases signal) → filename/title-text keywords ("sales"/"output" vs. "purchase"/"input").
- Per source: locate amount/VAT/category/reclaimable/description columns via a self-contained word-boundary synonym matcher (deliberately not Comparator's — different field set, lower-risk to duplicate ~15 lines than to reach into an unrelated shipped feature's tested code).
- Total rows (any row whose text contains "total") are excluded from per-row aggregation and instead compared against the computed sum — separately for a raw-total row vs. a reclaimable-only total row, since real registers print both (see §5's false-positive fix).
- Reclaimable-vs-blocked resolution per purchase row, per the priority order above — tier-1/tier-2 category keyword lists are grounded directly in Article 50 of the Saudi VAT Implementing Regulations (see §4).
- Any tier-2 items are collected across the WHOLE batch and surfaced together in one collective review step — never asked one row at a time.

4. Design decisions worth recording

- **One home-page card, two options — not two separate cards.** The two jobs are sequential stages of one real activity (checking invoices feeds preparing a return), and grouping by workflow rather than feature count already has precedent in this app (PDF Editor lives inside Converter's page for the same reason).
- **Batch-first, not wizard-first.** The old single-invoice correction modal was retired outright — a human correcting one invoice's fields before moving to the next doesn't hold up once real use is 20-30 invoices at once. Only what's flagged ever needs a person's attention.
- **Reclaimable-VAT categories are grounded in the real regulation text** (Article 50, "Goods and Services Deemed to be Received Outside of Economic Activity"), not general VAT-system knowledge: entertainment/sporting/cultural/catering/hotel/restaurant (Article 50(1)(a)-(b)) are tier-1 — blocked outright, no business-use exception in the text. Vehicles, their upkeep, and their fuel (Article 50(1)(c)-(e)) are tier-2 — Article 50(2) makes reclaimability genuinely conditional on work-exclusive or resale/rental-fleet use, which a keyword cannot determine.
- **"Business / client name", not "Customer name".** The old field name invited exactly the wrong mapping (buyer-on-one-invoice) for what it actually represents — whose filing this is, i.e. the seller/filer's own identity. Fixed at both the label and the value-mapping level so the ambiguity can't resurface.

5. Issues found via live testing, and their fixes

Every one of these was caught by actually running real invoices and the real test workbook through the live service — not assumed correct from code review or unit tests alone.

Nested event-loop crash

Found: The invoice-check endpoint was `async def`, but the calculation agent calls its own internal `asyncio.run()` — which cannot run inside an already-running event loop. Confirmed live: "asyncio.run() cannot be called from a running event loop".

Fixed: Made the endpoint a plain `def` — FastAPI runs those in a worker thread pool with no loop already running, matching every other agent-calling endpoint in this service.

Breaking dependency version

Found: `requirements.txt` pinned `mcp>=1.28.0` with no upper bound; pip silently resolved to `mcp 2.0.0`, a major version where the API this code was written against had moved — narration calls failed with `ModuleNotFoundError: No module named 'mcp.server.fastmcp'`.

Fixed: Pinned to `mcp>=1.28.0, <2.0.0`. Confirmed only this feature's two files use the package anywhere in the repo, so the fix is fully scoped.

Narration blind to structural issues

Found: An invoice flagged purely for a malformed VAT number got a narration reading "no mismatches found... correct" — because narration was generated from only the numeric-mismatch check, with zero visibility into structural issues computed separately. Individually accurate, jointly contradictory.

Fixed: Replaced with a single direct model call given the COMPLETE finding list — structural issues AND mismatches together — with an explicit instruction never to call anything correct/compliant while findings are listed.

Wrong extraction path for spreadsheets

Found: Period-return's extraction imported the `xlsx/csv` parser directly in-process, which needs `pandas` — only installed in doc-extraction's own container. Calculator is deliberately kept lightweight.

Fixed: Switched to calling doc-extraction over HTTP, the same pattern Job 1 already used — reuses the extraction SERVICE, not its heavy in-process dependencies.

False-positive consistency warning

Found: A real register can print BOTH a raw purchases total and a reclaimable-only total as two separate rows. Comparing the reclaimable-only total against the raw row sum flagged a false mismatch, even though both figures were individually correct.

Fixed: The two are now compared against the right figure each — a total row's own label text (containing "reclaimable" or not) decides which sum it's checked against.

PDF columns overflowing the page

Found: Reported directly from testing: PDF table columns appeared merged/not visible (XLSX was fine). Root cause: table column widths totaled 17cm against A4's ~15.9cm usable width with default margins — reportlab draws a table at exactly the widths given, it does not shrink to fit. Compounded by plain-string cells, which don't wrap — long filenames or reason sentences bled into the next column.

Fixed: Narrowed column widths to fit safely within the printable area, and wrapped every free-text cell in Paragraph objects so long text wraps within its own column. Verified by rendering the actual PDF to an image and inspecting it, not just re-reading the code.

Arabic PDF output — found broken, not yet fixed

Found: rendering every Arabic-language PDF variant to an image for visual inspection (not just checking the HTTP response) showed every Arabic character rendering as a solid black box, consistently across all four report types. Root cause: reportlab's default font has no Arabic glyphs at all — this is a complete rendering failure, not a subtle right-to-left shaping imperfection. English text, numbers, and static punctuation in the same documents render correctly; only Arabic-language content is affected.

Status: confirmed, not fixed. The real fix needs a bundled font with Arabic glyph coverage (e.g. Noto Naskh Arabic, OFL-licensed), registering it with reportlab, and proper Arabic text shaping/reordering (`arabic-reshaper` + `python-bidi`) before layout — new dependencies and a font asset, deliberately not done without confirming scope first, since the same unfixed pattern also affects Comparator's PDF reports.

6. Testing & verification

- 46 automated tests across the engine, structural validation, calculation tools, invoice-check, and period-return modules — all passing.
- Real ZATCA-shaped invoice XML run through the live service: a clean invoice, one with a malformed VAT number, and one with a genuine VAT/total mismatch — all three classified correctly with the right reasons.
- The real test workbook (a 3-sheet VAT register: sales, purchases, liability summary) run through the live service end to end — Output VAT, Reclaimable Input VAT, and Net position all matched the workbook's own printed Liability Summary figures exactly (net position -126.3, including correctly landing in a refund position), with zero false-positive consistency warnings.
- All three output formats (xlsx/pdf/docx) and both languages (en/ar) confirmed to generate successfully for both jobs — with the Arabic-rendering caveat above.

7. Known open items

- Arabic PDF rendering — confirmed broken, fix scoped but not started (see §5).
- The reclaimable/blocked category keyword lists cover Article 50(1)(a)-(e) directly, but 50(1)(f) — "any other goods/services used for a private or non-business purpose" — is a catch-all about actual usage, not a describable category, and can't be keyword-matched; nor can the Article 50(4) onward-resale exception be detected from expense-type text alone. Both rely on a human reviewing the workpapers' stated reason as the backstop.
- Not browser-tested by the assistant — no browser-automation tooling available in this environment; verification was via direct API calls against the live service plus rendering actual output files (PDFs) to images for visual inspection. The three new pages should be clicked through in a real browser before being treated as fully verified end to end.